LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 07 - ABSTRACT DAN INTERFACE



Jury Maxwell
103132400023
PS1DS-05-REG-01
PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Konsep abstraksi dan interface pada pemrograman berorientasi objek berfokus pada pemisahan antara rancangan spesifikasi sistem dengan detail kode implementasinya. Struktur kerangka kelas dapat disusun lebih awal tanpa perlu langsung menuliskan logika teknis secara mendetail di tahap awal perancangan. Pola semacam ini membuat standar komunikasi antar komponen program menjadi lebih jelas dan meminimalkan ketergantungan langsung antar kelas (loose coupling). Selain itu, berbagai objek yang berbeda bentuk tetap bisa dikelola secara seragam lewat mekanisme polimorfisme heterogen.

1. **Kelas Abstrak (Abstract Class)**: Merupakan kelas dengan kata kunci abstract yang tidak bisa dibuat menjadi objek secara langsung. Perannya lebih sebagai pondasi atau cetakan dasar untuk kelas-kelas turunannya. Di dalamnya tetap bisa diisi atribut umum, konstruktor, dan metode biasa yang sudah punya isi program, ditambah abstract method tanpa badan kode yang nantinya wajib dilengkapi sendiri oleh masing-masing subclass konkret.
2. **Interface**: Berisi kumpulan deklarasi metode abstrak yang bertindak sebagai kontrak perilaku bagi kelas mana saja yang menggunakannya. Berbeda dari pewarisan kelas biasa yang hanya mengizinkan satu induk (single inheritance), bahasa Java mendukung satu kelas untuk mengimplementasikan beberapa interface sekaligus (multiple realization). Kemampuan ini membuat objek-objek dari jalur silsilah kelas yang sama sekali berbeda tetap bisa dipproses menggunakan satu aturan yang seragam.
3. **Default dan Static Method pada Interface**: Pembaruan sejak Java 8 memperluas kemampuan interface agar tidak hanya berisi metode kosong. Lewat default method, sebuah metode di dalam interface sudah bisa memiliki isi program bawaan tanpa mengharuskan semua kelas pemakainya menulis ulang logika tersebut. Adapun static method disediakan sebagai fungsi bantuan (utility) yang langsung bisa dipanggil lewat nama interface tanpa perlu proses instansiasi objek.
4. **Polimorfisme Heterogen**: Teknik pengelompokan berbagai objek dengan tipe data berbeda ke dalam satu wadah koleksi yang sama, misalnya menggunakan array bertipe interface. Saat program dijalankan (runtime), metode yang sesuai pada tiap objek akan dieksekusi secara dinamis walaupun elemen-elemen di dalam array tersebut tidak berasal dari satu garis keturunan kelas yang sama.

B. Guided

1. Implementasi Dasar Kelas Abstrak

File: Employee.java

```
package guided.gd1;

public abstract class Employee {
    private String name;

    public Employee(String name) {
        this.name = name;
    }
```

```
    public String getName() {
        return name;
    }

    public abstract double earnings();
}
```

File: SalariedEmployee.java

```
package guided.gd1;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }

}
```

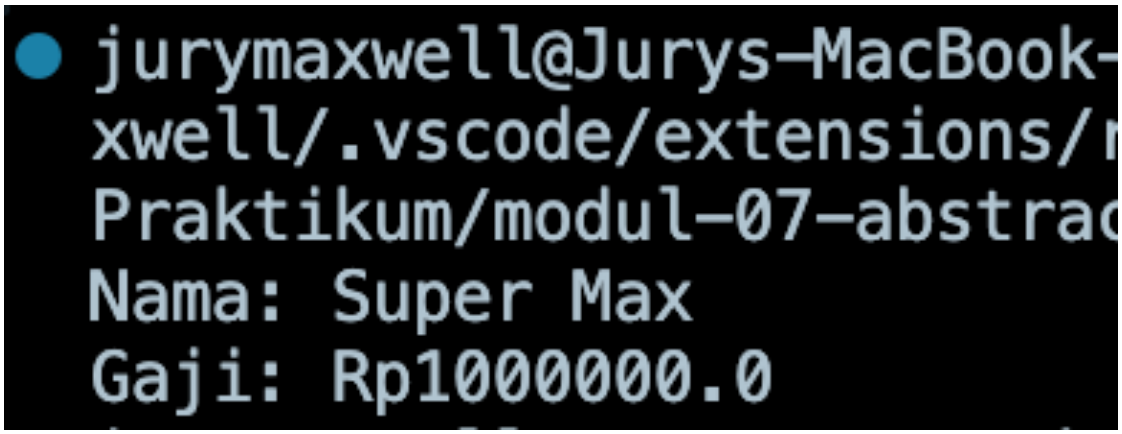
File: Main.java

```
package guided.gd1;

public class Main {

    public static void main(String[] args) {
        SalariedEmployee salaried = new SalariedEmployee("Super Max",
1000000);
        System.out.println("Nama: " + salaried.getName());
        System.out.println("Gaji: Rp" + salaried.earnings());
    }
}
```

Output :



2. **Penerapan Kontrak Interface dan Polimorfisme Heterogen**

File: Payable.java

```java
package guided.gd2;

public interface Payable {
    double getPaymentAmount();
}
```

File: Employee.java

```java
package guided.gd2;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

File: SalariedEmployee.java

```java
package guided.gd2;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }

}
```

File: Invoice.java

```java
package guided.gd2;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }

}
```

File: Main.java

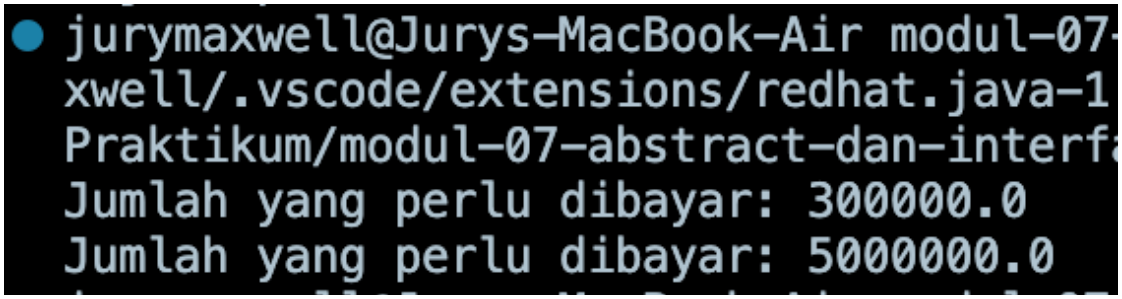
```java
package guided.gd2;

public class Main {
    public static void main(String[] args) {
       Payable[] items = new Payable[2];

       items[0] = new Invoice("Monitor LED", 2, 150000);
       items[1] = new SalariedEmployee("Super Max", 5000000);

       for (Payable item: items) {
           System.out.println("Jumlah yang perlu dibayar: " +
item.getPaymentAmount());
       }
    }
}
```

Output :



3. **Default Method dan Static Method pada Interface**

File: Payable.java

```java
package guided.gd3;


public interface Payable {
    double getPaymentAmount();

    // fungsi baru
    default void printReceipt() {
        System.out.println("Struk Resi - Total: Rp" +
getPaymentAmount());
    }

    // fungsi utilitas
    static void logTransaction(String message) {
        System.out.println("[AUDIT LOG] " + message);
    }
}
```

File: Employee.java

```java
package guided.gd3;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // every employee must implement earnings method but different
employees will have different implementations
    public abstract double earnings();

    // if there's a due payment, the amount will be the same as the
earnings of the employee
    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

File: SalariedEmployee.java

```java
package guided.gd3;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }

}
```

File: Invoice.java

```java
package guided.gd3;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }

}
```

File: Main.java

```java
package guided.gd3;

public class Main {
    public static void main(String[] args) {
        Payable.logTransaction("Sistem pembayaran diinisialisasi.");

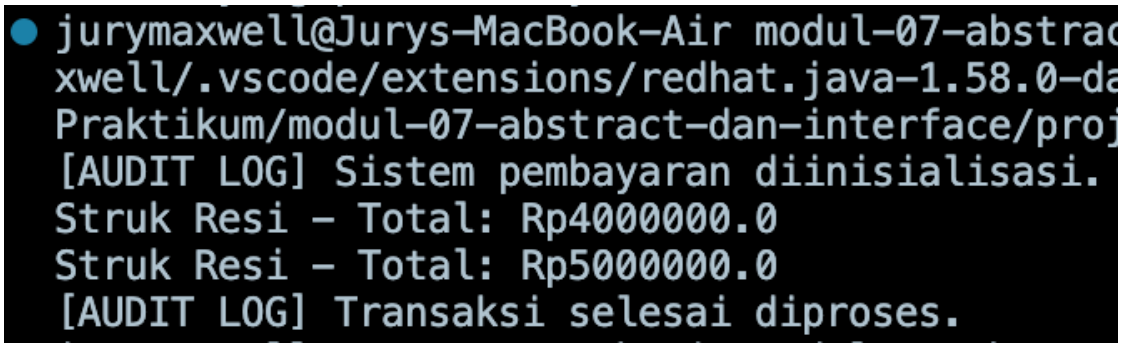
        Payable[] items = new Payable[2];
        items[0] = new Invoice("RAM DDR5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Super Max", 5000000);

        for (Payable item: items) {
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai diproses.");

    }
}
```

Output :



C. **Unguided**

1. **Sistem Kasir Terpadu Bioskop (Cinema POS)**

Sebuah bioskop memerlukan sistem kasir yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli Tiket Nonton, tetapi juga bisa membeli makanan atau minuman (Paket Snack F&B). Secara konsep arsitektur, Tiket Bioskop dan Paket Snack berasal dari garis keluarga hierarki yang sepenuhnya berbeda. Tiket film murni pun bersifat abstrak karena tidak bisa dibeli tanpa jenis studio yang jelas. Namun pada sistem kasir, tiket dan makanan harus dapat disatukan, dihitung akumulasi tagihannya, serta dicetak dalam satu struk otomatis.

Pengembangan sistem ini dibagi menjadi 4 tahapan tugas: membuat kelas abstrak TiketBioskop beserta subclass TiketReguler dan TiketVIP, merancang interface ItemTransaksi yang mengikat TiketBioskop dan PaketSnack, mengintegrasikan default method dan static method pada interface, serta menguji alur polimorfisme heterogen pada kelas Main menggunakan array statis.

File: ItemTransaksi.java

```
package unguided;

public interface ItemTransaksi {
    double getNominalBayar();
    String getNamaTransaksi();

    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi() + " | Total: Rp" +
getNominalBayar());
    }

    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

File: TiketBioskop.java

```java
package unguided;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int
jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return this.judulFilm;
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public int getJumlahKursi() {
        return this.jumlahKursi;
    }

    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();

    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

File: TiketReguler.java

```
package unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int
jumlahKursi, int nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    public int getNomorStudio() {
        return this.nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + this.nomorStudio + ") - " +
getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() +
")";
    }
}
```

File: TiketVIP.java

```
package unguided;

public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi,
double biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return this.biayaFasilitasVIP;
    }
```

```
    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + this.biayaFasilitasVIP) *
getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere – " + getJudulFilm() + " (" +
getJumlahKursi() + " x Rp" + (getHargaDasar() + this.biayaFasilitasVIP) +
")";
    }
}
```

File: PaketSnack.java

```
package unguided;

public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double
hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    public String getNamaPaket() {
        return this.namaPaket;
    }

    public int getJumlahPorsi() {
        return this.jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return this.hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return this.jumlahPorsi * this.hargaPerPorsi;
    }

    @Override
```

```
    public String getNamaTransaksi() {
        return "F&B Snack: " + this.namaPaket + " (" + this.jumlahPorsi +
" x Rp" + this.hargaPerPorsi + ")";
    }
}
```

File: Main.java

```
package unguided;

public class Main {
    public static void main(String[] args) {

        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk
Pelanggan #042.");

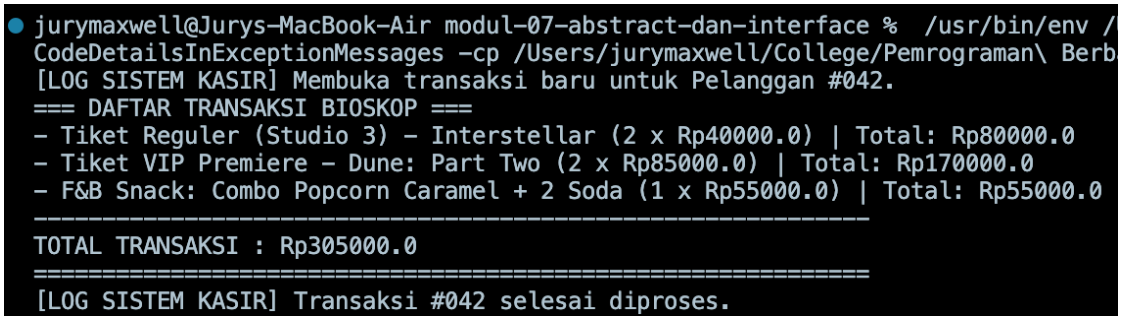
        ItemTransaksi[] items = {
            new TiketReguler("Interstellar", 40000, 2, 3),
            new TiketVIP("Dune: Part Two", 60000, 2, 25000),
            new PaketSnack("Combo Popcorn Caramel + 2 Soda", 1, 55000)
        };

        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double totalTransaksi = 0.0;
        for (ItemTransaksi item : items) {
            item.cetakRincianItem();
            totalTransaksi += item.getNominalBayar();
        }

        System.out.println("-----------------------------------------
-----------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + totalTransaksi);

System.out.println("=====================================================
========");
        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
    }
}
```

Output :



Penjelasan :

Sistem kasir bioskop (Cinema POS) ini menggabungkan pemesanan tiket film dan pembelian paket makanan ke dalam satu keranjang transaksi menggunakan kontrak *interface* ItemTransaksi. Pada struktur kelasnya, TiketBioskop dijadikan kelas abstrak berisi atribut judulFilm, hargaDasar, dan jumlahKursi guna mencegah pembuatan objek tiket tanpa tipe studio yang jelas. Kalkulasi harga tiket dikelola oleh kelas turunan TiketReguler serta TiketVIP yang memiliki tambahan biaya fasilitas per kursi, sedangkan kelas PaketSnack berdiri terpisah di luar hierarki tiket dengan perhitungan harga dari perkalian jumlah porsi dan harga satuan.

Seluruh objek tersebut disatukan ke dalam satu *array* statis bertipe ItemTransaksi pada kelas Main. Kehadiran *default method* cetakRincianItem pada *interface* membantu setiap objek mencetak rincian struknya secara langsung tanpa duplikasi kode, sementara *static method* catatLogKasir menangani pencatatan status transaksi. Alur program berjalan mulai dari pencatatan log pembuka, perulangan *array* untuk menampilkan rincian item sekaligus menjumlahkan total tagihan sebesar 305000, hingga penutupan transaksi melalui log kasir.

D. **Kesimpulan**

Berdasarkan hasil praktikum Modul 7 tentang Abstract Class dan Interface, kelas abstrak berperan sebagai cetakan dasar bagi kelas-kelas turunan yang memiliki kemiripan atribut umum tetapi membutuhkan detail perhitungan atau logika yang berbeda-beda. Larangan instansiasi langsung pada kelas abstrak turut menjaga keamanan struktur program agar tidak ada objek setengah jadi yang terbuat tanpa spesifikasi data yang lengkap.

Keberadaan interface melengkapi konsep abstraksi tersebut lewat mekanisme kontrak perilaku yang bebas diimplementasikan oleh kelas mana pun tanpa harus berada di satu pohon pewarisan yang sama. Fleksibilitas polimorfisme heterogen pun menjadi jauh lebih luas, contohnya saat menyatukan pemrosesan objek tiket bioskop dan paket makanan di dalam satu transaksi kasir. Dukungan default method dan static method pada versi Java modern juga sangat membantu ketika ada penambahan fungsi baru di level interface, sehingga kelas-kelas konkret yang sudah ada tetap berjalan stabil tanpa perlu banyak perombakan kode.